passgen

Deterministic Password Generator

Full documentation for project defense

1. What is passgen and why does it exist?

passgen is a program that creates passwords for you. But not just randomly - it does it in a clever way: it always creates the same password for the same website if you give it the same secret phrase.

Imagine you have accounts in ten different places: Google, VK, GitHub, Telegram, email, banking. If you use the same password everywhere - thats dangerous: if someone learns your password from one site, they can log in everywhere. If you use different passwords - how do you remember them all? Password managers like LastPass keep all passwords in one database, and if that gets hacked - everything is stolen.

passgen offers a third way: you remember just ONE phrase (anything, like "MyCatIsFluffy123"), tell the program the site name (like "google"), and the program gives you a complex password with mixed uppercase, lowercase, digits and maybe special characters. When you need that password again - you tell the program the same phrase and the same site name, and you get the same password. Nothing to store, everything in your head.

2. What is needed to run it

You only need Python 3 (version 3.6 or higher). On Linux (Arch, Ubuntu, etc.) it is usually already installed. On Windows you need to download it from python.org and during installation check "Add Python to PATH". No additional packages are required - everything works with the standard library.

How to launch:

1. Download the program. Either via git (git clone ...) or download ZIP from GitHub (green Code button -> Download ZIP) and extract.

2. Open a terminal. On Linux: cd xxxulay. On Windows: cd C:\path\to\folder.

3. Run: on Linux type python3 app.py, on Windows type python app.py. Press Enter.

4. The program window will open automatically. If not - the terminal will show an address like http://localhost:8765 - open it in any browser.

The window opens without an address bar, like a real desktop application. If this does not happen, it opens in a regular browser tab - it works the same way.

To close: press Ctrl+C in the terminal where the program is running.

3. How the program creates a password - the main algorithm

This is the most important question you will be asked: "How exactly does your program generate a password?" Here is the step-by-step explanation.

Step 1. User enters a secret phrase (seed)

The seed is any phrase the user must remember. For example "JohnIsCool228" or "ILoveCatsAndPython". The program converts this phrase into bytes (numbers 0-255). These bytes become the secret key for the cryptographic algorithm. Without this key, nobody can compute the password - not hackers, not anyone.

Step 2. The program forms a message for the algorithm

The program takes the service name (e.g. "google"), the desired password length (e.g. 16), and a step number (1, 2, 3...). All this is concatenated into one string. The step number exists because passwords can be longer than 32 characters - each step produces 32 bytes of new data, and if the password is longer, we do multiple steps.

Step 3. Computing HMAC-SHA256

HMAC-SHA256 is an international cryptographic standard. It is used in banking systems, in Bitcoin, in Wi-Fi protection, in software signing. It works like this: it takes a key (our seed), takes a message (service + length + step number), and produces 32 bytes that look like random garbage. Critical property: if the key and message are the same, the result is ALWAYS the same. If the key or message changes even slightly, the result changes completely.

Step 4. Converting bytes to characters

We have 32 bytes (numbers 0-255). We need to turn them into password characters. The user selected which character types to use: uppercase (A,B,C...), lowercase (a,b,c...), digits (0,1,2...), and/or special chars (!,@,#...). The program builds an alphabet from selected types. Then it takes each byte, checks if it is suitable (to avoid bias - all characters must appear equally often), and converts it to a character from the alphabet. If the password needs 16 characters but one step only produced 10 suitable bytes - the program does a second step, third step, etc., until the required length is reached.

Step 5. Additional options (inversion)

If the user enabled "First capital" - the program simply makes the first character uppercase. If "Last digit" is enabled - the last character is replaced with a digit (computed by a separate HMAC to preserve determinism). These options exist because many websites simply reject passwords without a capital letter and a digit.

Result: if a hacker learns your password for Google, they cannot log into your VK because the name is different ("vk" vs "google") and HMAC produces a completely different result. And if YOU forget the password - no problem: enter the same seed and same name, get the same password.

4. Why is the program considered secure?

Five arguments for why the program is safe to use:

1. Everything runs on your computer

The server runs on localhost (127.0.0.1 - your own machine). All data - seed, password, settings - travels only inside your computer. Nothing goes to the internet, nobody can intercept anything. The server is not connected to any external network.

2. Real cryptography is used

HMAC-SHA256 is not a homemade algorithm invented by students. It is an international standard described in RFC 2104 and FIPS PUB 180-4. Banks use it for transfers, Bitcoin uses it for addresses, Wi-Fi uses it for protection. The program does not invent anything - it just calls the built-in Web Crypto API in the browser. Like using a bank-grade safe instead of making a cardboard lock.

3. Password is not stored - nothing to steal

Password managers store all your passwords in one database. If hacked - everything is stolen. In passgen, the password is never stored. It is computed from scratch each time. Even the program does not know what password it will produce until it starts computing. Nothing is written to disk. Even settings (which services with which options) are not saved between launches - close the window, everything is gone.

4. Rejection sampling (uniform distribution)

When converting bytes to characters, the program uses a technique called rejection sampling: if a byte is outside the valid range, it is discarded and the next one is used. This ensures all characters appear with equal probability. Without this, some characters would appear more often and the password would be easier to guess.

5. Different passwords for different sites

The service name is part of the HMAC message. So the password for google is completely different from the password for github. If an attacker somehow learns the Google password, they cannot log into anything else. The program guarantees unique passwords automatically.

5. Every element on the screen explained

This section explains what each element in the program window is for. This will help you answer questions like "Why did you make this field this way?"

The "Seed" field

Enter your secret phrase here. This is the only thing you need to remember. Without it, the password cannot be generated - the program shows a pink error glow and a message. The error disappears as soon as you start typing. Pressing Enter in this field triggers password generation immediately.

The "Service" field

Enter the site name here: google, github, vk, yandex... Anything you want, even "my_favorite_forum_about_fish". This ensures different passwords for different sites. On the right there is a "list" button - click it to open a panel with popular services (Google, GitHub, VK, Yandex, Mail, iCloud, Dropbox, Telegram). Each has a colored circle logo for quick visual recognition. If your service is not in the list - just type it, the program will remember it. Right-click a saved service to delete it.

Password length

How many characters the password will have. Default is 10. Range: 1 to 99. Longer = more secure but less convenient to type. Arrows for quick adjustment. Recommended: 14-16.

Four circles - character sets

Toggle circles for uppercase, lowercase, digits, special characters. At least one must be selected. Special characters are off by default because some sites do not support them. Selection is remembered per service.

Presets dropdown

For those who do not want to configure manually. Three options: Minimal (8 chars, letters+digits), Standard (10 chars, everything except special), Maximum (16 chars, everything). Pick and generate.

Advanced settings (First capital, Last digit)

Hidden under an arrow to save space. "First capital" makes the first character uppercase. "Last digit" replaces the last character with a digit. Useful because many sites require a capital letter and a digit in passwords.

Login field and "Generate login" button

Some sites ask for a login too. The program generates a short 6-character login using only non-confusable letters (no i, l, o - they look like 1, 0). Each click produces a new login using an internal counter.

Password result field

The generated password appears here in monospace font. After generation, it is automatically copied to clipboard.

Strength bars

Three bars below the password. Pink (weak) -> crimson (medium) -> lilac (strong). Shows how strong the password is and gives a text recommendation.

The "Generate" button

The main button. Clicking it runs the entire HMAC-SHA256 algorithm, shows the result, copies it to clipboard, updates

strength bars, saves settings. Enter in the seed or service field works the same.

The "Copy" button

Copies the password to clipboard again if needed.

Export and Import

Small links at the top. Export downloads a JSON file with all settings. Import loads such a file back with a selection dialog - you choose which services to import.

6. Every function explained in detail

This section explains every JavaScript function in the code. These are the functions that make the program work. Each explanation is written so you can read it before the defense and explain everything confidently.

init - program startup

When the user opens the page, this is the first function that runs. Its job is to prepare everything. It clears old data from previous sessions (important for security - someone else might use the computer). Then it attaches event handlers: when the user presses Enter in the seed field, password generation should start. When the user types in the seed field, the error message should disappear. When the user clicks on a service in the list, that services settings should be restored. At the end, it draws the list of all available services (popular ones and those the user added before).

loadState - data cleanup for security

This function is called at the very start. It completely wipes all data: clears anything left in the browser from the last session, resets the service list, resets all settings to defaults (length 10, uppercase+lowercase+special chars, no inversion). This ensures each launch starts with a clean slate. If data were saved, someone else opening the program could see which services you use and what passwords you generate.

saveCfg - remembering settings for each service

When the user configures settings (length, character sets, inversion) and clicks "Generate", this function saves all these settings in the computers memory under the service name. For example: the user made a password for google with length 16 and all character sets - the function remembers: "google: length 16, all sets, no inversion". Next time the user selects google from the list, these settings are restored automatically. Important: settings are stored only in RAM while the program is open. Close the window - everything is gone. Much safer than saving to disk.

applyCfg - restoring settings for a service

This function does the reverse: it takes saved settings and applies them to the interface. For example, the user clicked on github in the service list. The function finds settings for github, sets the correct length, checks/unchecks the right character set circles, sets inversion options. The user immediately sees the settings they used last time and can either generate right away or tweak something.

toggleInv - toggling inversion options

When the user clicks on "First capital" or "Last digit", this function toggles that option. If it was off - turns it on, and vice versa. It immediately updates the appearance of the circle so the user sees the current state. A simple function but it provides a useful feature: many sites reject passwords without a capital letter and a digit, and these options guarantee the password will pass such checks.

applyPreset - quick configuration

The user selected "Minimal", "Standard" or "Maximum" from the dropdown. This function looks at the selection and sets ready parameters: for Minimal - length 8, letters+digits; for Standard - length 10, everything except specials; for Maximum - length 16, everything. Convenient for those who do not want to understand settings.

renderChk - drawing the character set circles

This function draws four rows with circles for selecting character sets. Each set is a row: circle and name ("Uppercase", "Lowercase", "Digits", "Specials"). If a set is active, the circle is filled purple. If not, just an outline. When the user clicks a row, the function runs again to update the display.

adj - adjusting password length

When the user clicks the up or down arrow next to the length field, this function adds or subtracts 1 from the current length. It ensures the length stays between 1 and 99. Thats all it does.

toggleSvcPanel - showing/hiding the service list

The service list panel is normally hidden to save space. When the user clicks "list" next to the service field, this function shows the panel with an opening animation. If the panel is already open, it closes it. The arrow next to the button rotates to indicate the current state.

renderSvcChips - building the service list

This function takes two lists: preset popular services (Google, GitHub, VK, Yandex, Mail, iCloud, Dropbox, Telegram) and user-added services, and builds a combined list. It shows popular services first (skipping those already in user list to avoid duplicates), then all user services. Each service becomes a clickable chip with a logo and name.

chip - creating one service button

This function is called for each individual service and creates one button. If the service is from the popular list - it makes a colored circle with a letter (Google = blue G, GitHub = dark GH, VK = blue V). Colors match real logos for easy recognition. If the service is user-added (e.g. "my_forum") - a gray circle with a question mark. When the user clicks the button - the service name fills the input field and settings are restored. Right-clicking a user service removes it from the list.

gen - THE MAIN FUNCTION, password generation

This is what the whole program was built for. When the user clicks "Generate" (or presses Enter), this function runs. It does many things in order:

1. Checks if the seed is entered. If not - shows a pink error glow and message, stops.

2. Checks if at least one character set is selected. If not - shows an error.

3. Builds an alphabet from selected character sets. For example, if uppercase and digits are selected: ABCDEFGHJKLMNPQRSTUVWXYZ0123456789.

4. Calls hmacGen function which computes HMAC-SHA256 and converts bytes to characters. The seed, service, length and alphabet are passed to it.

5. If "First capital" is enabled - makes the first character uppercase.

6. If "Last digit" is enabled - replaces the last character with a digit (computed by a separate HMAC so the same settings always give the same digit).

7. Displays the ready password in the result field.

8. Updates the strength indicator bars.

9. Copies the password to clipboard (so the user can paste it on the site immediately).

10. Adds the service name to the list if it was not there already.

11. Saves current settings for this service.

hmcGen - the cryptographic heart of the program

This function is the most important part for cryptography. It implements the HMAC-SHA256 computation. Here is what it does:

First, it takes the seed and turns it into an HMAC key. Then it forms a message: service name + password length + step number. Step starts at 0 and increases each time more data is needed.

Then it calls the browsers built-in function crypto.subtle.sign("HMAC", ...) - this is the standard cryptography built into every modern browser. The function returns 32 bytes (numbers 0-255).

For each byte, it checks: if the byte is below a certain threshold (max value divisible by alphabet length), it becomes a character from the alphabet. If the byte is too large - it is discarded (this is rejection sampling, ensuring uniform distribution).

If one step did not produce enough characters, step number increases by 1 and the next HMAC step is computed. This repeats until the required number of characters is collected. Then the function returns the finished password.

genLogin - login generation

Many sites ask for a login. This function generates a simple 6-character login. It works similarly to password generation but simpler: it computes SHA-256 (just a hash, not HMAC) from: seed + "login" + service + counter. The counter increases with each button press - so each click produces a new login. From the first 6 hash bytes, characters are made. The login alphabet is special: only lowercase letters (but without i, l, o - they look like 1, 0) and digits (2-9, without 0 and 1 for the same reason). The login is short and easy to remember.

updateStrength - evaluating password strength

After the password is generated, this function looks at its length and the number of selected character sets and decides how good it is. Three levels: weak (pink) - if length < 10 or < 3 sets; medium (crimson) - if length >= 10 and 3+ sets; strong (lilac) - if length >= 14 and all 4 sets. It also shows a text recommendation.

cpy - copying password to clipboard

Tries to copy the password to clipboard. First tries the modern way (navigator.clipboard.writeText). If that fails (old browser), uses the old way (execCommand). Shows "Copied!" message for 1.5 seconds.

exportData - exporting settings

Collects all services and their settings into one object, converts to JSON format (text with indentation for readability) and downloads file "passgen_export.json" to the users computer. The file contains: which services exist and what settings they have.

importData - importing settings

User selects a JSON file that was previously exported. This function reads the file, verifies it is valid JSON and contains passgen data. If everything is ok - shows a selection window with all services found in the file.

showImportModal - show the service selection dialog

Creates and shows a dialog window over the program, listing all services from the imported file. Each service has a checkbox, all enabled by default. The user can uncheck services they do not want to import. Buttons at bottom: "Import" and "Cancel".

confirmImport - confirm and execute import

When the user clicks "Import" in the dialog, this function collects checked services. For each checked service, it loads settings into the program and adds it to the service list. If the service already existed - settings are updated. Shows "Import completed!" message for 3 seconds.

closeImportModal - cancel import

If the user changed their mind and clicked "Cancel" - the function simply closes the dialog and clears temporary data. No settings are changed.

toggleAdvanced - show/hide advanced settings panel

When the user clicks on "Advanced" with the arrow, this function either expands the panel with inversion options (first capital, last digit) or collapses it back. The arrow rotates to show the state.

7. Export and Import - why are they needed?

Export and Import let you transfer settings from one computer to another. For example, you configured the program at home on your desktop: added 15 services, each with its own settings. Now you go to university with a laptop and want the same settings there. Click "Export" at home - a file passgen_export.json is downloaded. Transfer it to the laptop (via USB, email, whatever), open the program on the laptop, click "Import", select the file - all settings are transferred. No need to reconfigure everything.

The file looks like text that says: here is the list of services, and here are the settings for each. The format is called JSON - a standard way to write data so both computers and humans can read it.

When you import, the program does not just import everything blindly. First it shows a list: "these services are in the file, which ones do you want to import?" You check only the ones you need. Useful when the file has 30 services but you only need 2.

8. Quick summary - how to answer "What does your program do?"

If asked to explain the essence briefly:

passgen is a program that creates passwords not randomly, but based on a secret phrase and the service name. It uses the cryptographic algorithm HMAC-SHA256 which guarantees that the same input (seed + service + settings) always produces the same password. The password is not stored - it is computed from scratch each time. You only need to remember one phrase to have unique complex passwords for all your accounts. The program is written in Python (server) and JavaScript (interface with cryptography). All computation happens locally on the users computer, nothing is sent over the internet.

The program can:
- Generate passwords using HMAC-SHA256
- Remember separate settings for each service
- Show password strength with colored bars
- Generate login names
- Copy passwords to clipboard automatically
- Export and import settings as JSON
- Run in session mode (no data saved to disk)